

Qualitative Data Analysis



Dirk Riehle, FAU Erlangen

NYT B03

Licensed under [CC BY 4.0 International](https://creativecommons.org/licenses/by/4.0/)

Course Content and Structure

Overview

1. What is science?
2. Scientific research
3. **Qualitative data analysis**

Theory building

4. Theory building
5. Systematic reviews
6. Qualitative surveys
7. Action research
8. Case study research

Theory validation

9. Theory validation
10. Survey research
11. Controlled experiments

Comprehensive

12. Design science

Academia

13. Academic writing
14. Academic publishing

Agenda

1. QDA in theory building
2. QDA artifacts
3. QDA methods
4. Straussian coding
5. Quality assurance
6. Iterative process
7. Theory presentation

1. QDA in Theory Building



Qualitative Data Analysis (QDA) [1]

Qualitative data analysis is a research method

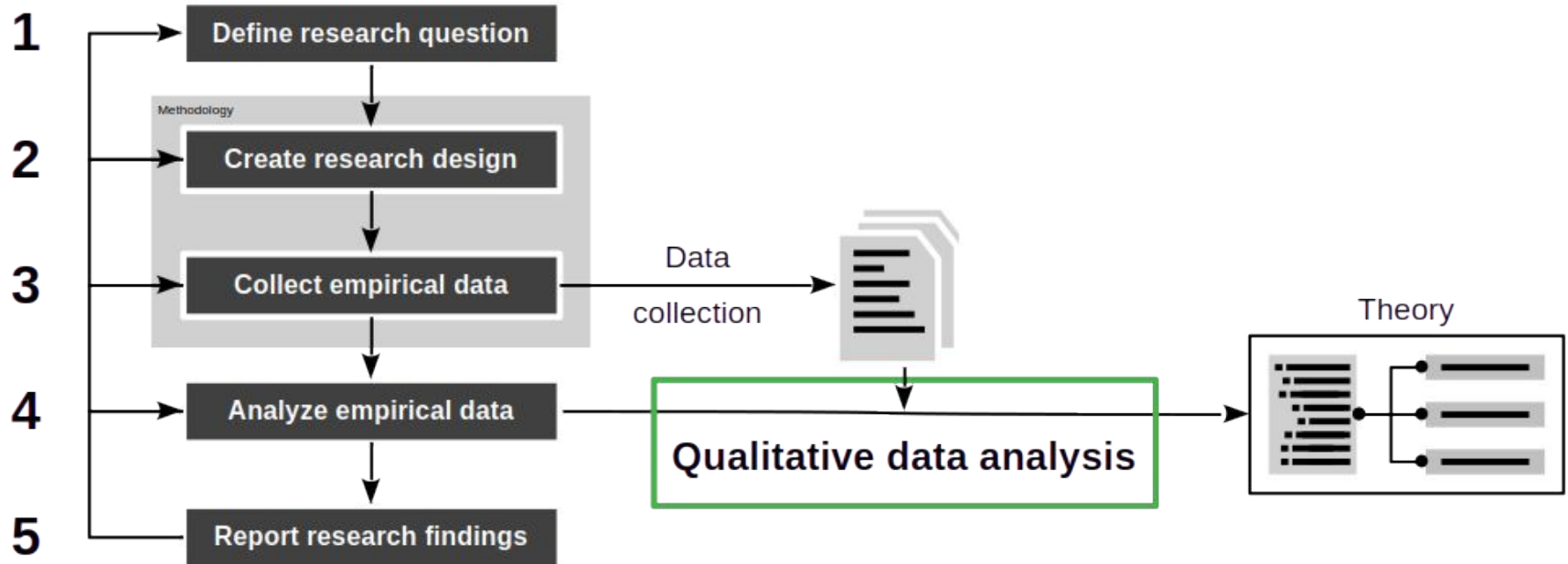
- For deriving meaning from qualitative empirical data
- In response to one or more research questions

Typically used in theory building and development

- Inductive use → theory building
- Deductive use → theory evaluation

Position and Purpose of QDA in Theory Building

DR



QDA in Theory Building Methodologies

Most QDA methods can be plugged into theory building methodologies

- Systematic reviews
- Qualitative surveys
- Action research
- Case study research

Properties of QDA

The qualitative data analysis process is

- Inductive (→ theory building from data)
- Structured (→ not arbitrary / random)
- Quantitative (→ measurable quality)

2. QDA Artifacts



Artifacts and Activities in QDA

DR

Data Analysis

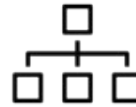
Data collection
(primary materials)



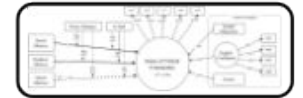
Data extraction
(codes and codings)



Data synthesis
(themes / categories)



Theory
presentation



Codes and Codings

A code (also label, annotation, etc.) is

- A short text (name) for an empirical observation

An open code is a code that

- Originates from the primary materials through a coding

A coding is

- The relationship between an open code and its origin in the data

A theme or category is a code that

- Hierarchically abstracts from other codes in a code system

Example Code and Codings

QDAcity

Search...

Project Dashboard

Coding

Editor

Code

Statistic

No collaborators

Documents

10JNIO-1.PDF

11 Spohrer, J. C., & Soloway, E. (1984).PDF

12DENN-1.PDF

13ETTL-1.PDF

14MCCA-1.PDF

29YARY-1.PDF

15ALBR-1.PDF

25KACZ-1.PDF

Code System

Code System

Domain-Specific Errors

Environment and Language Misco...

Logical errors

Memory Management Errors

Misinterpretation

Semantic Errors

Sloppiness/Typographical Errors

Strategic Errors

Syntax Errors

Incorrect Loop Syntax

Incorrect Operators

Invalid Keywords Usage

Missing Semicolons

String Comparison

Unbalanced Delimiters

Variable not declared or initialized

Investigating Novice Programming Mistakes:
Educator Beliefs vs Student Data

Neil C. C. Brown and Amjad Altadmri
School of Computing
University of Kent
Canterbury, Kent, UK
{nccb,aa803}@kent.ac.uk

ABSTRACT

Educators often form opinions on which programming mistakes novices make most often – for example, in Java: “they always confuse equality with assignment”, or “they always call methods with the wrong types”. These opinions are generally based solely on personal experience. We report a study to determine if programming educators form a consensus about which Java programming mistakes are the most common. We used the Blackbox data set to check whether the educators’ opinions matched data from over 100,000 students – and checked whether this agreement was mediated

use of a single equal sign to compare equality” [7]. “A common mistake in for loops is to accidentally put a semicolon at the end of the line that includes the for statement” [4]. In this paper, we set out to determine: are such observations accurate in general? This matters for several reasons. Firstly, recent research by Sadler et al [15] suggests that knowledge of student misconceptions is important to educator efficacy, and thus it is of interest whether educators’ impressions about student mistakes are correct. Secondly, when educators communicate with each other (face-to-face or online), it matters whether their opinions on common

Codings

Code Properties

Code Relationship

Code Memo

Code Book Entry

ID	Document	Author
7680431649508616580	3BROWN-1.PDF	Max Neuwinger
1566122804253825610	3BROWN-1.PDF	Max Neuwinger
-8604540218958801331	3BROWN-1.PDF	Max Neuwinger
-7929572003070382130	8 Hristova, M., Misra, A., Rutter, M., & Mercuri, R. (2003). Identifying and correcting Java programming errors for introductory computer science students.pdf	Max
-5403255529159963536	8 Hristova, M., Misra, A., Rutter, M., & Mercuri, R. (2003). Identifying and correcting Java programming errors for introductory computer science students.pdf	Max

Feedback

Code Definition

A code definition should include

- A short (one sentence) definition
- A description of
 - When to use the code and
 - When not to use the code

Memo

A memo is

- A written reflection on a code

A memo may include

- Research insights and observations
- Work notes for future investigations

The code itself should be defined independently of the memo

Example Code and Memo

The screenshot displays the QDAcity web application interface. The top navigation bar includes a search bar, a notification bell, and a user profile icon. The main interface is divided into a left sidebar for the 'Code System' and a central workspace for editing code and memos.

Code System Sidebar:

- Code System (0)
- Domain-Specific Errors (1)
- Environment and Language Misconceptions (7)
- Logical errors (5)
- Memory Management Errors (0)
- Misinterpretation (1)
- Semantic Errors (7)
- Sloppiness/Typographical Errors (7)
- Strategic Errors (0)
- Syntax Errors (14)
 - Incorrect Loop Syntax (5)**
 - Incorrect Operators (16)
 - Invalid Keywords Usage (4)
 - Missing Semicolons (5)
 - String Comparison (4)
 - Unbalanced Delimiters (21)
 - Variable not declared or initialized (13)

Central Workspace:

- Name:** Incorrect Loop Syntax
- Author:** Max Neuwinger
- Color:** Blue
- Codemap description:** (Empty text area)
- Codemap labels:** Search or create label...
- Definition:** Errors in the syntax of loops.
- When To Use:** Example: Using commas instead of semicolons in for loops
- When Not To Use:** (Empty text area)
- Coded Text Segments:**
 - ☒ Select All
 - ☐ 3BROWN-1.PDF PDF
 - ☐ "A common mistake in for loops is to accidentally put a semicolon at the end of the line that includes the for statement" [4].
 - ☐ E : Incorrect semi-colon after an if selection structure before the if statement or after the for or while repetition structure before the respective for or while loop. For example: if (a == b); return 6;
 - ☐ F : Wrong separators in for loops (using commas instead of semi-colons) For example: for (int i = 0, i < 6, i++) ...
 - ☐ 2 8 Hristova, M., Misra, A., Rutter, M., & Mercuri, R. (2003). Identifying and correcting Java programming errors for introductory ...

Code Memo:

Incorrect loop syntax makes the execution of the program fail.

This type of error, like syntax errors in general, should be caught by a compiler.

Note: Investigate relationship between static and dynamic typing for novices.

Feedback

Save unsaved changes

Code System



A code system is

- A hierarchical structure of codes intended
- To capture the relationships between the codes

A code system is an expression of the theory under development

- Codes represent concepts and constructs
- Relationships represent hypotheses

Example Code System

The screenshot displays the QDAcity web application interface. On the left, a tree view under 'Code System' lists various error categories, with 'Incorrect Loop Syntax' selected at the bottom. The main area is divided into several sections: 'Name' (Incorrect Loop Syntax), 'Author' (Max Neuwinger), 'Color' (a blue bar), 'Codemap description' (empty), 'Codemap labels' (a search bar), 'Definition' (Errors in the syntax of loops.), 'When To Use' (Example: Using commas instead of semicolons in for loops), 'When Not To Use' (empty), 'Coded Text Segments' (a list of references with a 'Select All' checkbox), and 'Code Memo' (a text box with a 'Feedback' button). The top navigation bar includes 'Project Dashboard', 'Coding', 'Editor', 'Code', and 'Statistic' tabs, along with a search bar and a user profile icon.

Code System

- Code System (0)
- Domain-Specific Errors (1)
 - Dataset Misunderstandings (4)
 - Faulty Data Analysis Logic (12)
 - Incorrect Data Handling (4)
 - Misunderstanding Data Types (2)
- Environment and Language Misconceptions (7)
 - Incorrect Language Understanding (34)
 - Library Misuse (2)
 - Path and File Errors (3)
- Logical errors (5)
 - Faulty Conditional Logic (2)
 - Incorrect Loop Conditions (4)
 - Off-by-One / Out-of-Bounds Errors (12)
- Memory Management Errors (0)
 - Memory Language Misconceptions (4)
 - Memory Leaks (1)
- Misinterpretation (1)
 - Incorrect Programming Assumptions (11)
 - Task Misunderstanding (10)
- Semantic Errors (7)
 - Incorrect Function Usage (12)
 - Type Mismatches (24)
 - Variable Scope Issues (6)
- Sloppiness/Typographical Errors (7)
- Strategic Errors (0)
 - Suboptimal Coding (12)
 - Wrong Algorithm (13)
- Syntax Errors (14)
 - Incorrect Loop Syntax (5)**

Name: Incorrect Loop Syntax

Author: Max Neuwinger

Color: [Blue bar]

Codemap description:

Codemap labels: Search or create label...

Definition: Errors in the syntax of loops.

When To Use: Example: Using commas instead of semicolons in for loops

When Not To Use:

Coded Text Segments:

☒ Select All

☐ 3BROWN-1.PDF PDF

☐ "A common mistake in for loops is to accidentally put a semicolon at the end of the line that includes the for statement" [4].

☐ E : Incorrect semi-colon after an if selection structure before the if statement or after the for or while repetition structure before the respective for or while loop. For example: if (a == b); return 6;

☐ F : Wrong separators in for loops (using commas instead of semi-colons) For example: for (int i = 0, i < 6, i++) ...

☐ 2 8 Hristova, M., Misra, A., Rutter, M., & Mercuri, R. (2003). Identifying and correcting Java programming errors for introductory

Code Memo:

Incorrect loop syntax makes the execution of the program fail.

This type of error, like syntax errors in general, should be caught by a compiler.

Note: Investigate relationship between static and dynamic typing for novices.

Save unsaved changes

Feedback

The Semantics of Hierarchy

In principle, anything goes, but ...

Common patterns of hierarchical organization are

- A slotting system to group similar open codes (cf. card sorting)
- A grouping system for underlying themes (cf. thematic analysis)
- A structuring mechanism for theoretical constructs / abstractions

Computer science students may use common modeling techniques like

- Association, aggregation (part-whole), generalization/specialization

No tool (yet) allows for the formal capture of such (meta) relationships

Code Book

A code book documents the code system

For each code in the code system, the code book lists

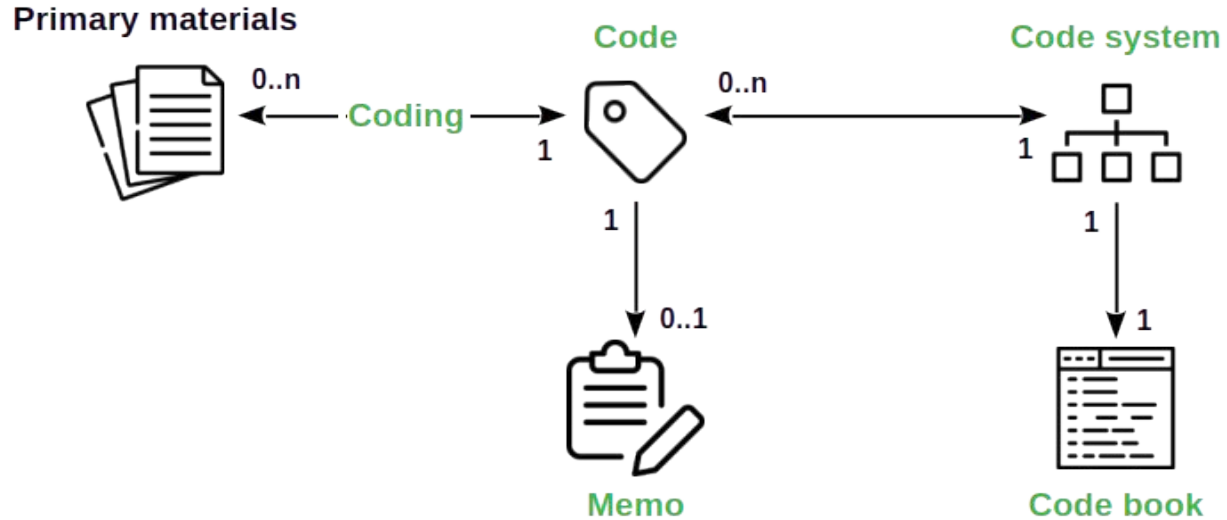
- The definition of the code
- Instructions for when to use the code
- Possible disambiguations towards other codes
- An exemplary piece of data coded with the code

Beyond general documentation, a code book

- Helps establish intercoder reliability of analysis

Main Artifacts in Overview

DR



3. QDA Methods



Three Different QDA Methods

1. Card sorting [1]
2. Thematic analysis [2]
3. Straussian coding [3]

[1] See [Hudson \(2013\)](#): Card sorting.

[2] See [Braun & Clarke \(2012\)](#): Thematic analysis.

[3] See [Corbin & Strauss \(2008\)](#): Qualitative research, 3rd edition.

Card Sorting [1]

Analyze data

- Extract data
 - Name item [2] and write on card
- Synthesize data
 - Group cards with similar items

Codify theory

- Name and summarize groups

[1] See [Hudson \(2013\)](#): Card sorting.

[2] Item, topic, theme, artifact, ... depends on context and is relative to research question

Coding, Then

Research interview number 6. Semi-structured interview with N.N., teacher.

The researcher (Question): What governs your day-to-day work?

Interviewee (Answer): The headmaster, the manager of our school, has a set of rules and regulation. I can talk to some of my colleagues about anything, if I have a question about something. For example, the other day we discussed the importance of locating, and to fix problems. And to do it fast. !!

Handwritten notes: Rulebook (Updating?), To fix Located & fix, Talking to colleagues

The researcher (Question): This year, do you have any particular goal, when it comes to your job as a teacher at this school?

Interviewee (Answer): Not really. I mean, I want my pupils to learn as much as possible. So that is a goal, I guess. Sometimes it is hard, though. (Short silence.) They keep changing the schedules. The management, I mean. So frustrating. (Silence.) And all the meetings. I wish I not have to attend all those.

Handwritten notes: Changing schedules, Attending meetings (but why not?)

The researcher I see. Other than that?

And Now (With QDAcity)

QDAcity

Search...

Project Dashboard

Coding Editor Code Statistic

No collaborators

Documents

10JNIO-1.PDF

11 Spohrer, J. C., & Soloway, E. (1986).PDF

12DENN-1.PDF

13ETTL-1.PDF

14MCCA-1.PDF

15ALBR-1.PDF

1SINGH-1.PDF

25KACZ-1.PDF

Code System

Code System

Domain-Specific Errors

Environment and Language Misco...

Logical errors

Memory Management Errors

Misinterpretation

Semantic Errors

Sloppiness/Typographical Errors

Strategic Errors

Syntax Errors

Incorrect Loop Syntax

Incorrect Operators

Invalid Keywords Usage

Missing Semicolons

String Comparison

Unbalanced Delimiters

Variable not declared or initialized

Unbalanced Delimiters

Incorrect Operators

Incorrect Loop Syntax

Incorrect Loop Syntax

Unbalanced Delimiters

Invalid Keywords Usage

Type Mismatches

Unbalanced Delimiters

Syntax Errors

at how long students take to solve different errors, Dy and Rodrigo [6] looked at improving the error messages given to students, and Ahmadzadeh et al. [1] looked at student error frequencies and debugging behaviour. Jackson et al. [10] identified the most frequent errors among their novice programming students. All six of these studies used compiler error messages to classify errors. However, early results from McCall [14] suggest that compiler error messages have an imperfect (many-to-many) mapping to student misconceptions. Additionally, all six studies looked at cohorts of (up to 600) students from a single institution.

Our study is novel in that it looks at student mistakes from a much larger number of students (over 100,000) from a large number of institutions¹, thus providing more robust data about error frequencies. An earlier paper about Black-box gave a brief list of the most frequent compiler error messages [3], but in this study we do not simply use compiler error messages to classify errors. Instead, we borrow error classifications from Hristova et al. [9], which are based on surveying educators to ask for the most common Java mistakes they saw among their students.

2.2 Educators' Opinions

Spohrer and Soloway [17] proposed in 1986 to examine the accuracy of educator folk wisdom. However, they did not survey educators, and instead took two anecdotal folk wisdoms about learning to program and then compared them to actual data from students (in Pascal). Ben-David Kolkant [2] interviewed some educators about students' ap

C: Unbalanced parentheses, curly brackets, square brackets and quotation marks, or using these different symbols interchangeably.
For example: `while (a == 0)`
D: Confusing "short-circuit" evaluators (`&&` and `||`) with conventional logical operators (`&` and `l`).
For example: `if ((a == 0) & (b == 0)) ...`
E: Incorrect semi-colon after an if selection structure before the if statement or after the for or while repetition structure before the respective for or while loop.
For example:
`if (a == b);`
`return 6;`
F: Wrong separators in for loops (using commas instead of semi-colons)
For example: `for (int i = 0, i < 6, i++) ...`
G: Inserting the condition of an if statement within curly brackets instead of parentheses.
For example: `if {a == b} ...`
H: Using keywords as method names or variable names.
For example: `int new;`
I: Invoking methods with wrong arguments (e.g. wrong types).
For example: `list.get("abc")`
J: Forgetting parentheses after a method call.
For example: `myObject.toString;`
K: Incorrect semicolon at the end of a method header.
For example:
`public void foo();`

Codings

Code Properties

Code Relationship

Code Memo

Code Book Entry

ID	Document	Author
7680431649508616580	3BROWN-1.PDF	Max Neuwinger
1566122804253825610	3BROWN-1.PDF	Max Neuwinger
-8604540218958801331	3BROWN-1.PDF	Max Neuwinger
-7929572003070382130	8 Hristova, M., Misra, A., Rutter, M., & Mercuri, R. (2003). Identifying and correcting Java programming errors for introductory computer science students.pdf	Max
-54032555291599635	8 Hristova, M., Misra, A., Rutter, M., & Mercuri, R. (2003). Identifying and correcting Java programming errors for introductory computer science	Max

Feedback

Thematic Analysis [1]

Analyze data

- Extract data
 - Familiarize yourself with the data
 - Generate initial codes
- Synthesize data
 - Search for potential themes
 - Review potential themes
 - Define and name themes

Codify theory

- Produce the report

Straussian Coding [1]

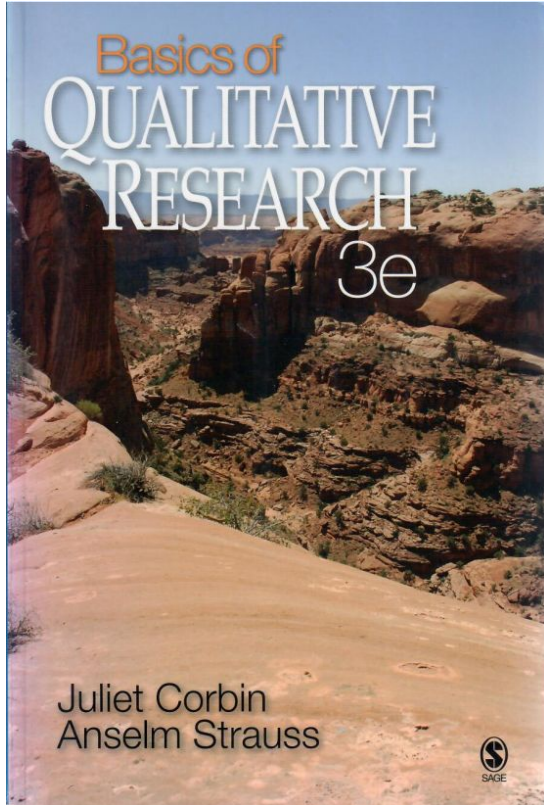
Analyze data

- Extract data
 - Perform open coding
- Synthesize data
 - Perform axial coding
 - Perform selective coding

Codify theory

- Write articles

Grounded Theory



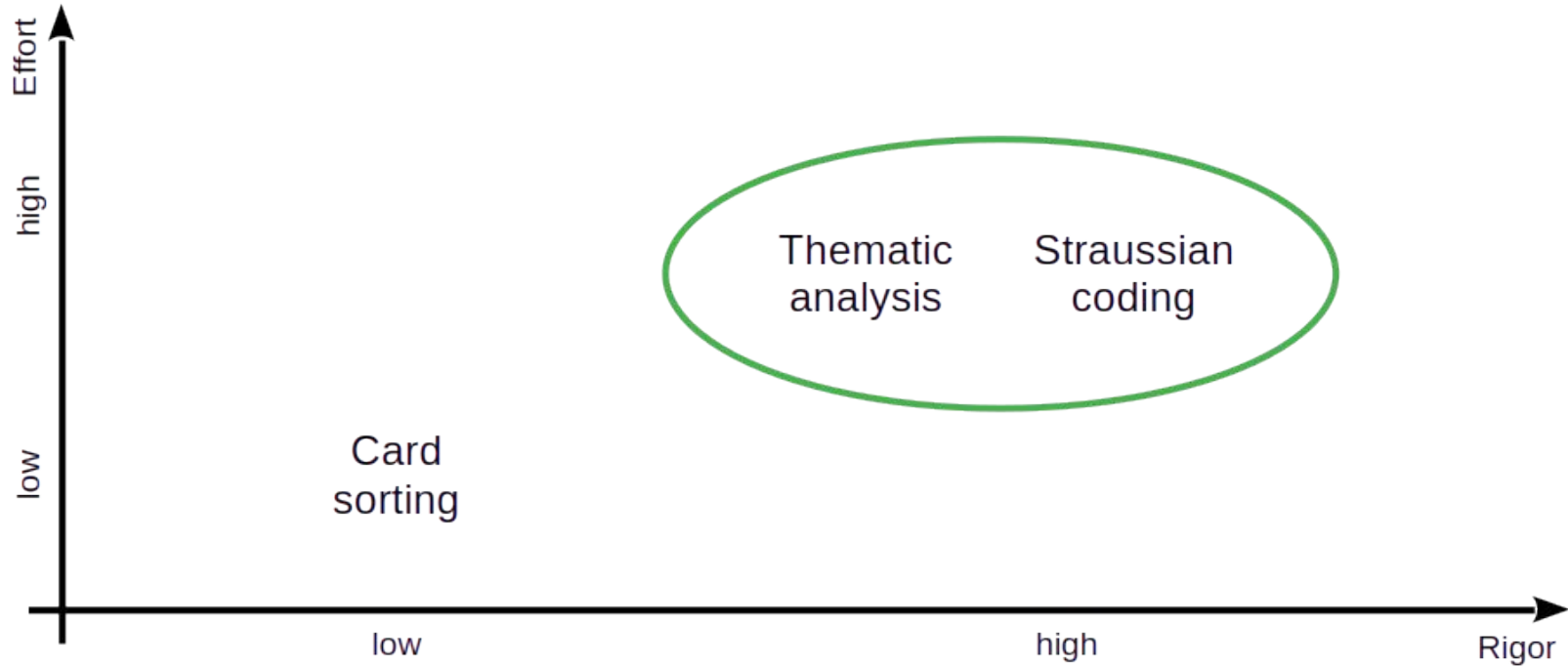
Grounded theory is a research methodology for theory building like qualitative surveys, etc.

Straussian coding was lifted from Corbin & Strauss' version of grounded theory [1]

Artifacts and Activities in QDA Methods

		Card sorting	Thematic analysis	Straussian coding
Artifacts	Codes	Items	Codes	Open codes
	Categories	Item groups	Themes and subthemes	Concepts and categories
Activities	Extraction	Card writing	Initial coding	Open coding
	Synthesis	Grouping and summarizing	Theme naming and defining	Axial and selective coding

Rigor vs. Effort in QDA Methods



A Word of Caution

Despite my correlation and attempted homogenization

- Method authors and users can be extremely picky

So in a given research project, always use the specific method

4. Straussian Coding



Open Coding 1 / 2

An open code is a code that

- Represents an empirical observation
- Is conceptual, i.e. represents a concept
- Has a direct origin in the primary materials

Open coding is

- The creation of open codes from primary materials

An in-vivo code is an open code that

- Has a name taken directly from the primary materials

Open Coding 2 / 2

Carefully read the primary materials incrementally, repeatedly

Mark segments containing relevant information for your research question

Assign segment to existing open code or create a new one

Write memos for codes to elaborate on your observations

Constant Comparison

As you are coding, compare codes

- Are two codes the same?
- Are there subtle differences? Should the codes be split?

Then take action and refactor your code system

- Implies rereading and recoding

Axial Coding

Axial coding is

- The introduction of new codes representing concepts or constructs by
- Comparing existing codes for similarities and differences

Axial coding creates a hierarchy of codes, the code system

Codes should be MECE that is

- Mutually Exclusive and
- Completely Exhaustive

Concepts and Constructs

Codes represent concepts; examples

- Off-by-one error (open code)
- Syntax error (axial code)

Codes represent constructs; examples

- Product owner / software developer conflict (open code)
- Performance incentive (axial code)

Selective Coding

Selective coding is

- The focussing of top-level codes on the research question and
- The relationship-building of top-level codes to a single core code

According to Corbin & Strauss, the theory is the single core code

Resulting Theory

Code systems and associated artifacts represent the theory

There is more to the theory in a researcher's mind

It needs research articles to document this

5. Quality Assurance



General Criteria for Qualitative Research

Quality criteria based on Guba & Lincoln [1]

- Credibility
- Transferability
- Dependability
- Confirmability

Typically realized through specific practices

[1] See [Guba & Lincoln \(1982\)](#): Naturalistic inquiry.

Example Research Practices

Two example research practices

- Member checking
- Peer debriefing

Fairly simple and easy

Member Checking

Member checking is a research practice in which the researcher

- Seeks feedback from data sources (participants, stakeholders) either
- During data gathering (active and reflective listening) or
- After analysis (asking for feedback and confirmation)

Peer Debriefing

Peer debriefing is a research practice in which the researcher

- Subjects their work to review and critique by a peer
- At multiple times during the current running project

The researcher presents and explains their work

The peer questions and critiques the work, looking for flaws

The researcher documents the session with a protocol

QDA-Specific Concerns

How do you know you are done?

⇒ Theory saturation

How do you know you are not winging it?

⇒ Intercoder agreement

Theory Saturation [1]

Theory saturation is a

- Stopping criterion for qualitative data analysis

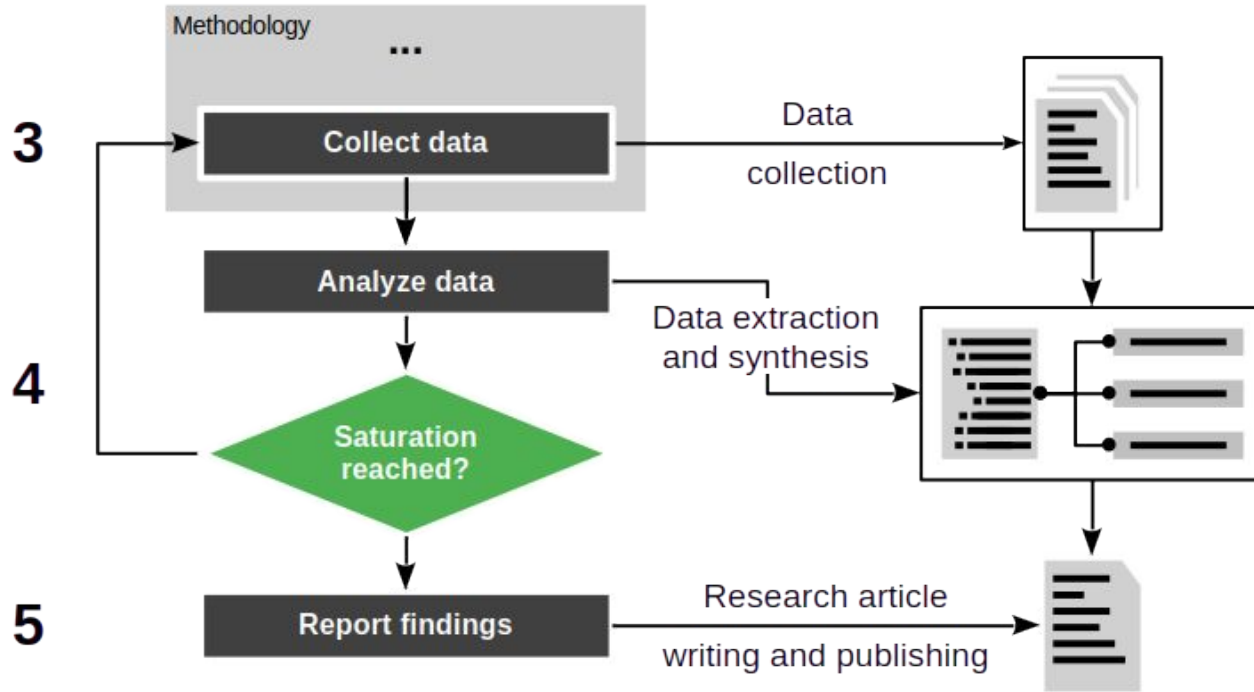
Theory saturation is reached when

- Further data collection and analysis does not yield new insights

In measurable terms, the change rate of codes is zero

- New data yields new codings, but no new codes

Theory Saturation in Research Process



Measuring Theory Saturation

Structure analysis into iterations

- Use purposive sampling to seek out new data

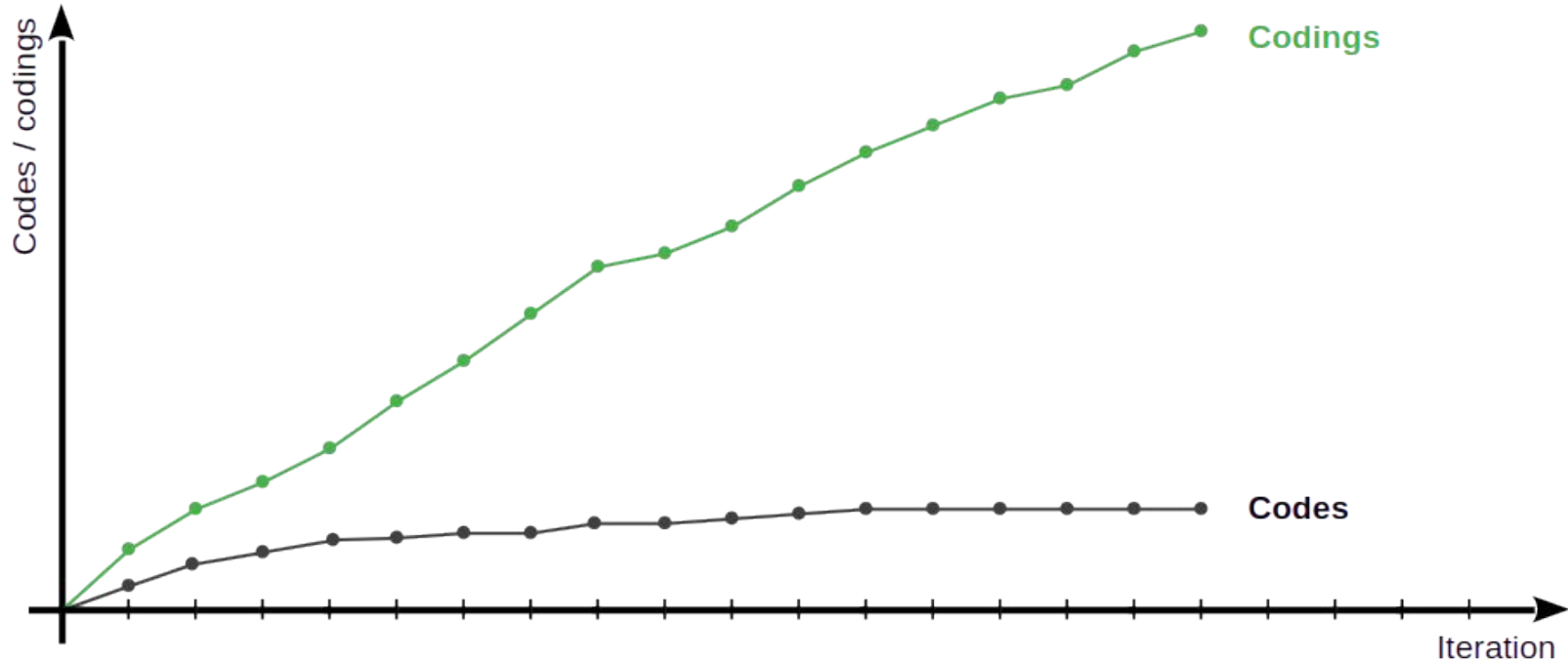
After finishing the analysis of new data

- Count the number of new codings and codes

No new codes in five iterations

- Theory saturation reached

Charting of Theory Saturation by Iteration



Intercoder Reliability

Intercoder reliability (also: interrater reliability) is

- The degree of agreement between
 - A primary (the original) coder and
 - One or more secondary coders

In other words: Is there intersubjective agreement?

- And to what extent?

Process of Measuring Inter coder Agreement

The original coder creates the code book

- The code book documents the code system

The original coder explains and discusses the code book with further coders

- Further coders (usually one) code the primary materials using the code system

The intercoder agreement is the overlap/agreement of the coding sets

DR

Codes and codings (second coder)

- Cohen's Kappa (κ)
- Krippendorff's Alpha (α)
- Scott's Pi
- Fleiss' Kappa

6. Iterative Process



Theory Building is Iterative

Theory building proceeds in iterations

- Iterations incrementally answer the primary research question at hand
- Triangulation and expansion improve rigor and increase relevance

Research Project Iteration

A research project proceeds in iterations

- Each iteration provides an increment
- Stopping criterion is theory saturation
- Each iteration gathers new data based on insights
- New data (samples) follow sampling model and strategy

The research project determines the primary methodology

Build-out Using Multiple Research Projects

Theory building proceeds incrementally, across many research projects

- Larger research projects are composed of smaller ones
- Multiple research projects allow for triangulation (rigor)
- Multiple research projects expand scope (relevance)

Each research project may use a different methodology

- Systematic review
- Qualitative survey
- Action research
- Case study research

7. Theory Presentation



Theory Presentation

- Prose-based descriptions
- Mathematical models / equations
- Structural equation models [1]
- Concept matrices [2]
- Handbooks [3]

[1] See [Palvia et al. \(2006\)](#): Research models.

[2] See [Webster & Watson \(2002\)](#): Analyzing the past to prepare for the future.

[3] See [Riehle et al. \(2025\)](#): Pattern discovery and validation (a.k.a. the handbook method).

Summary

1. QDA in theory building
2. QDA artifacts
3. QDA methods
4. Straussian coding
5. Quality assurance
6. Iterative process
7. Theory presentation

Thank you! Any questions?



dirk.riehle@fau.de – <https://oss.cs.fau.de>

dirk@riehle.org – <https://dirkriehle.com> – [@dirkriehle](#)

Legal Notices

License

- Licensed under the [CC BY 4.0 International](https://creativecommons.org/licenses/by/4.0/) license

Copyright

- © 2012-2026 Dirk Riehle, some rights reserved